SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

CO5 ASSESSMENT TOOL 2 – Seminar on Emerging Technologies

Course Code:	CSA1011	**Course Title:**	Software Engineering
CO Assessed:	CO5	**Assessment:**	Assessment Tool 2 – **Seminar on Emerging Technologies**
Weightage:	20%	**Total Marks:**	25
CO5 Statement:	Evaluate emerging technologies and societal impacts, maintaining and evolving software systems responsibly		
Student Name:	AKTCHAYA A	**Reg. No.:**	192424096
Date:	27/08/2026	**Duration:**	60 minutes

Code of Conduct

I AKTCHAYA A certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student:AKTCHAYA A

Annexure A – Seminar on Emerging Technologies

Instructions to Students:

Students must select the topic **strictly according to the Serial Number (S.No.) order** assigned to them.

S.No.	Seminar Topic
1	**Generative AI and Large Language Models (LLMs)**
2	**AI Agents and Autonomous AI Systems**
3	**Multimodal Artificial Intelligence**
4	**Explainable and Trustworthy AI**
5	**Edge AI and Intelligent Edge Computing**
6	**Federated Learning and Privacy-Preserving AI**
7	**Deep Learning for Computer Vision**
8	**AI-Powered Medical Image Analysis**
9	**Natural Language Processing with Transformer Models**
10	**Retrieval-Augmented Generation (RAG)**
11	**AI in Cybersecurity and Threat Detection**
12	**Zero Trust Architecture and Modern Cybersecurity**
13	**Post-Quantum Cryptography**
14	**Quantum Computing and Quantum Algorithms**
15	**Quantum Machine Learning**
16	**Cloud-Native Computing and Microservices**

17	**Serverless Computing and Function-as-a-Service**
18	**Edge Computing and Fog Computing**
19	**Cloud Security and Confidential Computing**
20	**Digital Twins and Intelligent Simulation**
21	**Internet of Things (IoT) and Smart Systems**
22	**Industrial Internet of Things (IIoT)**
23	**5G/6G Networks and Next-Generation Communication**
24	**Blockchain Technology and Decentralized Applications**
25	**Web3 and Decentralized Internet**
26	**Smart Contracts and Blockchain Applications**
27	**Augmented Reality (AR) and Virtual Reality (VR)**
28	**Extended Reality (XR) and Immersive Technologies**
29	**Robotics and Autonomous Robots**
30	**Human–Robot Interaction and Collaborative Robots**
31	**Autonomous Vehicles and Intelligent Transportation Systems**
32	**Smart Cities and AI-Driven Urban Technologies**
33	**Metaverse Technologies and Virtual Worlds**
34	**Brain–Computer Interfaces (BCI)**
35	**Wearable Computing and Smart Wearable Devices**
36	**Green Computing and Sustainable IT**
37	**AI for Software Engineering and Code Generation**
38	**DevSecOps and Intelligent CI/CD Pipelines**
39	**Digital Forensics and AI-Based Cybercrime Investigation**
40	**Privacy-Enhancing Technologies and Data Protection**

Seminar Presentation Structure:

1. **Introduction to the Emerging Technology**
2. **Need and Motivation**
3. **Working Principle / Architecture**
4. **Key Components and Technologies**
5. **Applications / Real-World Use Cases**
6. **Advantages and Limitations**
7. **Recent Developments and Trends**
8. **Challenges and Future Scope**
9. **Case Study / Example**
10. **Conclusion and References**

Rubrics for Calculation

Evaluation Criteria	Excellent (4 Marks)	Good (3 Marks)	Satisfactory (2 Marks)	Needs Improvement (1 Mark)
Technical Content & Subject Knowledge (25%)	Comprehensive and accurate content; demonstrates excellent understanding of the emerging technology	Good technical understanding with minor gaps	Basic understanding with limited technical depth	Incomplete content or major technical errors
Organization & Presentation Structure (15%)	Excellent logical flow with clear introduction, content, conclusion, and effective transitions	Well-structured with minor organizational issues	Adequately structured but some sections lack clarity	Poorly structured and difficult to follow
Examples, Diagrams & Case Study (20%)	Excellent use of relevant diagrams, examples, applications, and real-world case studies	Good use of relevant examples and diagrams	Limited examples/diagrams with basic explanation	Inappropriate or insufficient examples/diagrams
Communication & Presentation Skills (20%)	Very clear, confident, engaging delivery with excellent eye contact, voice modulation, and time management	Clear and confident presentation with minor issues	Understandable but lacks confidence or engagement	Unclear delivery, excessive reading, or poor time management
Question Handling & Overall Performance (20%)	Answers questions accurately and confidently; demonstrates strong overall knowledge	Answers most questions correctly	Answers basic questions with some difficulty	Unable to answer questions or demonstrate adequate understanding

Criteria	Mark
Technical Content & Subject Knowledge (25%)	/5
Organization & Presentation Structure (15%)	/5
Examples, Diagrams & Case Study (20%)	/5
Communication & Presentation Skills (20%)	/5
Question Handling & Overall Performance (20%)	/5
TOTAL	/25

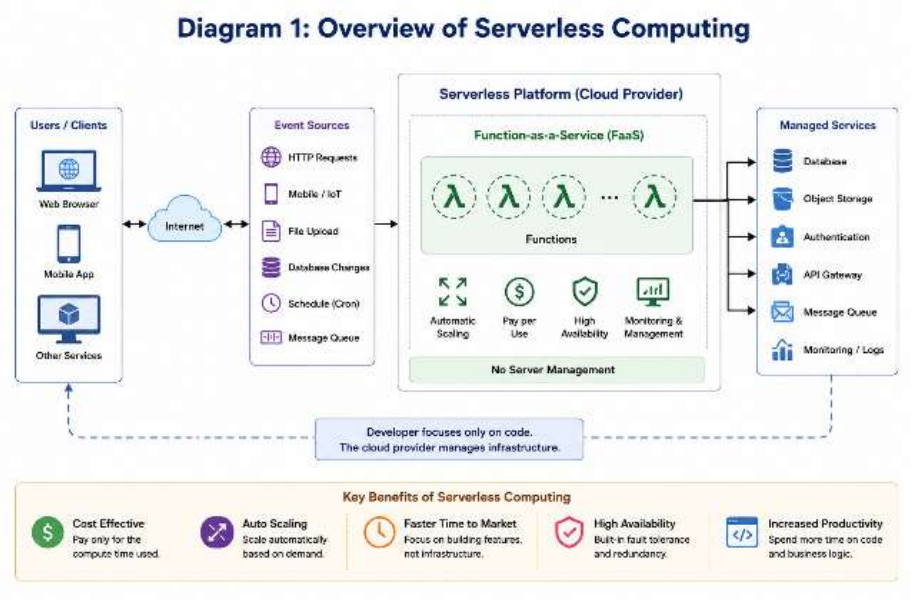
SERVERLESS COMPUTING AND FUNCTION-AS-A-SERVICE (FaaS)

1. Introduction

Cloud computing has changed the way software applications are developed, deployed, and managed. Traditionally, developers had to manage physical servers, virtual machines, operating systems, networking, and other infrastructure components. Serverless computing provides an alternative approach in which cloud providers manage most of the underlying infrastructure while developers focus primarily on application logic.

Serverless computing is a cloud execution model where applications can run without the developer directly managing servers. One of the most important models of serverless computing is **Function-as-a-Service (FaaS)**. FaaS allows developers to deploy individual functions that are executed in response to events such as HTTP requests, database updates, file uploads, or scheduled tasks.

Serverless computing and FaaS have become important technologies for building scalable, event-driven, and cost-efficient applications. Major cloud providers offer serverless platforms, including AWS Lambda, Microsoft Azure Functions, and Google Cloud Functions.



2. Objectives

The main objectives of serverless computing and FaaS are:

1. To reduce the need for infrastructure management.
2. To allow developers to focus on application functionality.
3. To automatically scale applications according to demand.
4. To reduce operational and infrastructure costs.
5. To support event-driven application development.
6. To provide rapid application deployment.
7. To improve resource utilization through on-demand execution.

3. Serverless Computing

Serverless computing is a cloud computing model in which the cloud provider dynamically manages the servers required to execute an application.

The term **serverless does not mean that servers are not used**. Servers are still required to run applications, but their provisioning, maintenance, scaling, patching, and availability are handled by the cloud provider.

In traditional application development, a developer or organization may have to:

- Purchase or provision servers.
- Install operating systems.
- Configure networking.
- Manage storage.
- Apply security patches.
- Monitor infrastructure.
- Configure scaling.
- Maintain server availability.

With serverless computing, most of these responsibilities are transferred to the cloud provider.

Basic Serverless Architecture

User → API Gateway → Serverless Function → Database/Storage

For example, when a user uploads an image, an event can automatically trigger a serverless function. The function can process the image and store the result in cloud storage.

4. Function-as-a-Service (FaaS)

Function-as-a-Service is a cloud computing model that allows developers to execute small, independent pieces of code called **functions** without managing the underlying servers.

A function normally performs a specific task. It is triggered by an event and runs only when required.

Examples of events include:

- HTTP requests
- File uploads
- Database changes
- Messages from a queue
- Scheduled events
- IoT device events
- User actions

A typical FaaS execution follows these steps:

1. An event occurs.
2. The cloud platform detects the event.
3. The required function is invoked.
4. The function executes the required operation.
5. The result is returned or stored.
6. The computing resources are released or reused.

5. Difference Between Serverless Computing and FaaS

Serverless computing is a broader concept, while FaaS is one of its major implementation models.

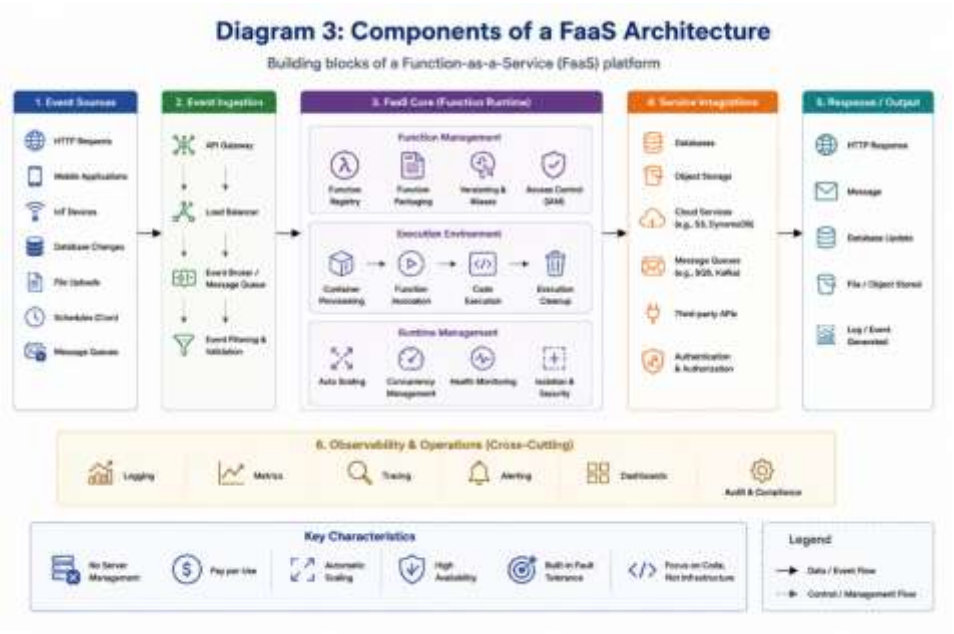
	Function-as-a-Service
Broad cloud computing approach	Specific serverless execution model
Includes multiple managed services	Mainly focuses on functions
Infrastructure is managed by provider	Infrastructure for function execution is managed by provider
Can include serverless databases, storage, APIs, etc.	Executes individual pieces of code
Supports complete serverless architectures	Used as a building block of serverless applications

Therefore, **FaaS can be considered an important component of serverless computing**.

6. Architecture of FaaS

A typical FaaS architecture contains the following components:

- **Client:** The client generates a request or performs an action that starts the application process.
- **API Gateway:** An API Gateway receives HTTP or REST requests and forwards them to the appropriate serverless function.
- **Function:** The function contains the application logic. It performs a specific operation when triggered.
- **Event Trigger:** The trigger determines when a function should execute. It can be an HTTP request, database event, file upload, or scheduled event.
- **Cloud Storage:** Storage services can be used to store files, images, logs, or other application data.
- **Database:** Serverless applications can communicate with managed databases to store and retrieve application information.
- **Monitoring System:** Cloud providers provide monitoring and logging services to track function executions, errors, execution time, and resource consumption.

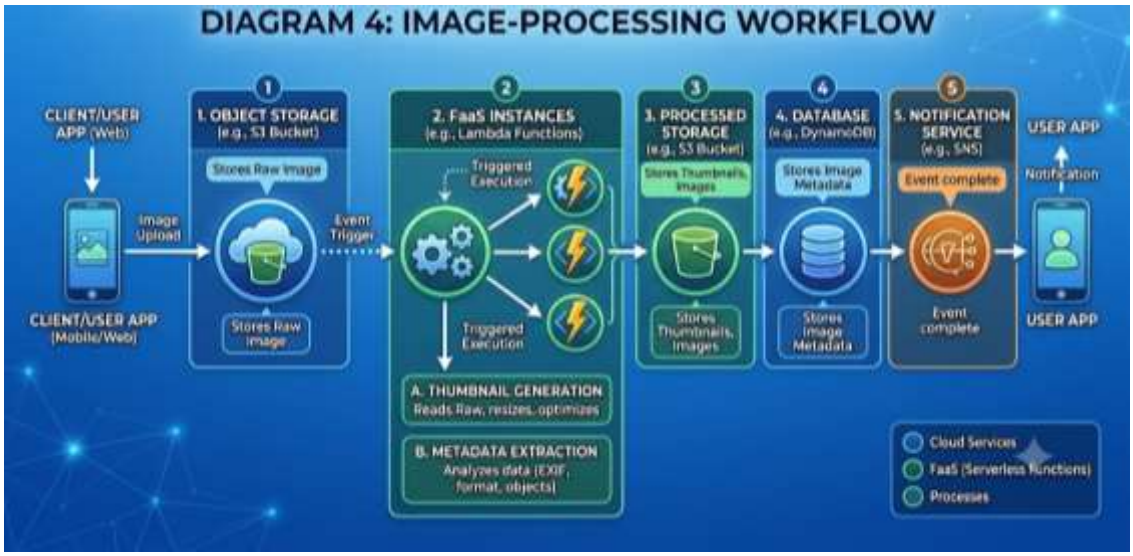


7. Working of Function-as-a-Service

Consider an online image-processing application.

1. A user uploads an image.
2. The image is stored in cloud storage.
3. The upload generates an event.
4. The event triggers a serverless function.
5. The function processes the image.
6. The processed image is stored in another location.
7. The user can access the processed image.

The developer does not need to manually create a server for every image-processing request.



8. Major FaaS Platforms

Several cloud providers offer FaaS services.

- AWS Lambda

  AWS Lambda is Amazon Web Services' serverless compute service. It executes code in response to events and automatically manages the computing resources required to run the code.

- Azure Functions

  Azure Functions is Microsoft's serverless compute service. It supports event-driven execution and integrates with other Azure services.

- Google Cloud Functions

  Google Cloud Functions allows developers to execute functions in response to events generated by cloud services, HTTP requests, and other triggers.

- Other Platforms

  Other serverless technologies and platforms include:

  - Cloudflare Workers
  - IBM Cloud Functions
  - OpenFaaS
  - Knative

9. Programming Languages Used in FaaS

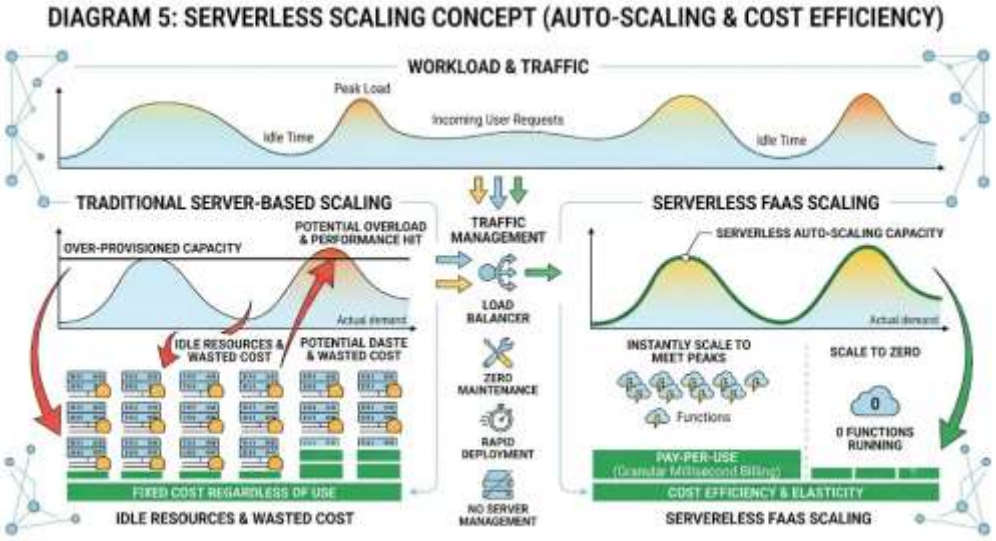
FaaS platforms generally support several programming languages. Depending on the cloud provider, developers can use languages such as:

- Python
- JavaScript
- TypeScript
- Java
- Go
- C#
- Ruby

Python and JavaScript are commonly used for serverless applications because they are suitable for developing lightweight event-driven functions.

10. Advantages of Serverless Computing

- **No Server Management:** Developers do not have to directly manage servers, operating systems, or infrastructure.
- **Automatic Scaling:** Serverless platforms can automatically increase or decrease computing resources according to workload.
- **Cost Efficiency:** Users generally pay according to resource consumption and function execution rather than maintaining continuously running servers.
- **Faster Development:**Developers can concentrate on application logic instead of infrastructure configuration.
- **Event-Driven Architecture:**Serverless platforms are well suited for applications that respond to events.
- **High Availability:**Cloud providers typically manage infrastructure availability and redundancy.
- **Easy Deployment:**Functions can be deployed independently, making application updates faster and more manageable.



11. Disadvantages and Challenges

Although serverless computing provides several benefits, it also has limitations.

- **Cold Start:** A function that has not been recently used may require additional time to initialize before execution. This delay is known as a **cold start**.
- **Execution Limits:** FaaS platforms may impose limits on execution duration, memory, package size, or other resources.
- **Vendor Lock-In:** Applications designed specifically for one cloud provider may become difficult to migrate to another provider.
- **Debugging Complexity:** Debugging distributed serverless applications can be more complicated because multiple functions and cloud services may interact.
- **Monitoring Challenges:** Applications may contain many independent functions, making logging, tracing, and monitoring more complex.
- **Security:** Although the cloud provider manages infrastructure security, developers are still responsible for securing function code, permissions, APIs, and application data.

12. Serverless vs Traditional Cloud Computing

Feature	Traditional Cloud	Serverless
Server management	Required	Managed by provider
Scaling	Usually configured by user	Mostly automatic
Billing	Often based on provisioned resources	Usually based on actual usage
Deployment	Application/server based	Function/service based
Maintenance	Higher	Lower
Infrastructure control	Higher	Lower
Development speed	Moderate	Faster for suitable workloads
Best suited for	Long-running applications	Event-driven workloads

13. Applications of Serverless Computing

Serverless computing is used in many real-world applications.

- **Web Applications:**Serverless functions can process API requests and provide backend functionality for web applications.
- **Mobile Applications:**FaaS can provide backend services for mobile applications, such as authentication, notifications, and data processing.
- **Data Processing:**Functions can process large amounts of data when events occur.
- **Image and Video Processing:**Uploaded media files can automatically trigger functions for resizing, compression, conversion, or analysis.
- **IoT Applications:**IoT devices can generate events that trigger serverless functions for processing sensor data.
- **Chatbots:**Serverless functions can process messages and generate responses for chatbot applications.
- **Scheduled Tasks:**Functions can execute periodically for tasks such as generating reports, cleaning data, or sending notifications.
- **Real-Time Notifications:**Events can trigger functions that send emails, alerts, or push notifications.

14. Example: Serverless Student Management Application

A simple student management system can use serverless architecture.

Architecture:

User → API Gateway → FaaS Function → Database

For example:

- A student submits registration details.
- The request reaches the API Gateway.

- The API Gateway invokes a function.
- The function validates the student information.
- The function stores the information in a database.
- A response is returned to the user.

Separate functions can be created for:

- Adding students
- Updating student details
- Deleting students
- Searching students
- Generating student reports

This approach allows each operation to be developed and deployed independently.

15. Security in Serverless Computing

Security is an important consideration in serverless applications.

Important security practices include:

- Use strong authentication and authorization.
- Apply the principle of least privilege.
- Protect API endpoints.
- Encrypt sensitive data.
- Secure environment variables and secrets.
- Validate user input.
- Monitor function activity.
- Maintain updated dependencies.
- Implement proper logging.
- Regularly review cloud permissions.

Serverless security follows a **shared responsibility model**, where the cloud provider secures the underlying infrastructure while the application developer is responsible for securing application code, configurations, access permissions, and data.

16. Performance Considerations

Performance in FaaS depends on several factors:

- Function initialization time
- Memory allocation
- Execution duration
- Network latency
- Database access
- Number of function invocations
- External API dependencies

Developers can improve performance by keeping functions small, reducing unnecessary dependencies, optimizing database access, and selecting appropriate resource configurations.

17. Serverless and Microservices

Serverless computing and microservices are related but different concepts.

Microservices divide an application into small, independent services. Serverless FaaS can be used to implement individual components of a microservices architecture.

For example:

User Service → User Function

Payment Service → Payment Function

Notification Service → Notification Function

Each function can perform a specific operation and communicate with other services through APIs or messaging systems.

18. Future Scope

Serverless computing is expected to continue developing as cloud platforms improve their performance, security, monitoring, and developer tools.

Future developments may include:

- Improved cold-start performance
- Better distributed tracing
- Greater support for AI and machine learning workloads
- Improved serverless databases
- Edge-based serverless execution
- Better multi-cloud support
- More efficient event processing
- Integration with IoT and real-time systems

Serverless technologies are also expected to play an important role in modern application architectures because organizations increasingly require scalable and flexible cloud applications.

19. Conclusion

Serverless computing provides a modern approach to cloud application development by reducing the need for developers to manage infrastructure directly. Function-as-a-Service is one of the key technologies behind serverless architectures, allowing applications to execute small pieces of code in response to events.

The major benefits of serverless computing include automatic scaling, reduced infrastructure management, faster development, and usage-based resource consumption. However, challenges such as cold starts, execution limitations, vendor lock-in, monitoring complexity, and security responsibilities must be considered.

Overall, serverless computing and FaaS are useful technologies for developing scalable, event-driven, and cost-efficient applications. They are particularly suitable for APIs, data processing, automation, IoT systems, mobile backends, and event-driven applications.

20. References

1. Amazon Web Services – AWS Lambda Documentation.
2. Microsoft Azure – Azure Functions Documentation.
3. Google Cloud – Cloud Functions Documentation.
4. Cloud Native Computing Foundation – Serverless Computing Resources.
5. IBM Cloud – Serverless Computing Resources.
6. OpenFaaS Documentation.
7. Research literature on serverless computing and Function-as-a-Service architectures.